

Manual técnico de implementación — Preajuste de PdV según el origen de la orden

Módulo a construir: `pos_preset_by_order_origin`
Cliente que lo motiva: Pastelería del Jardín
Versión objetivo: Odoo 19.0 (Enterprise + `odoo-pro`)
Documento previo: [Evaluación de viabilidad](#)
Fecha: 31 de agosto de 2026

Este manual describe **todo lo necesario para implementar** el requerimiento: código completo, puntos de integración con el core, configuración de datos, pruebas, entorno de desarrollo y checklist de entrega. Está escrito para que un desarrollador que no participó en el análisis pueda ejecutarlo de principio a fin.

1. Requerimiento en una frase

El preajuste (`pos.order.preset_id`) debe deducirse del **origen** de la orden: si la orden tiene mesa (`table_id`) → *Comer en el local*; si no la tiene → *Para llevar*; y si una venta directa pasa a una mesa, el preajuste debe recalcularse.

2. Prerrequisitos

2.1 Conocimiento técnico

Área	Nivel necesario
ORM de Odoo (modelos, herencia, <code>res.config.settings</code>)	medio
Frontend del PdV: OWL + <code>patch()</code> sobre servicios/modelos	imprescindible
Modelos relacionales del PdV (<code>this.models["pos.order"]</code> , <code>pos.load.mixin</code>)	medio
Tours JS (<code>web_tour.tours</code>) y tests Hoot unitarios	medio
Docker del entorno <code>dev_env_odoo_pro-19</code>	básico

2.2 Módulos y entorno

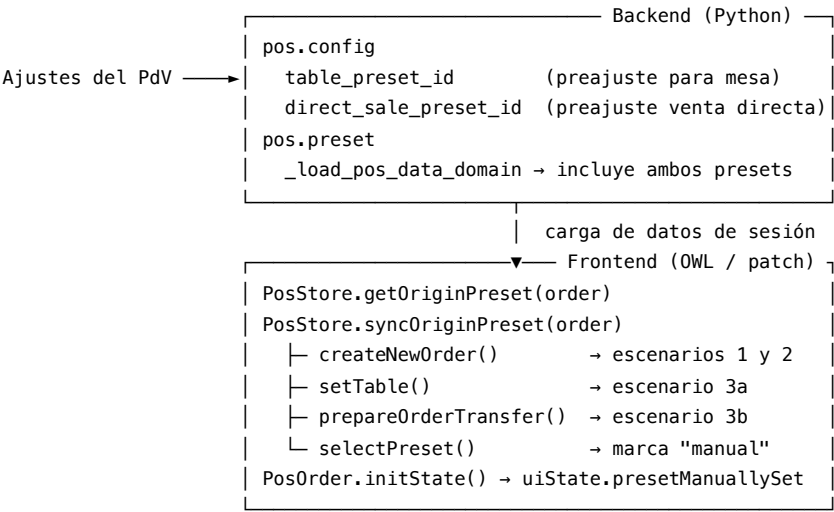
- Módulos requeridos instalados: `point_of_sale` , `pos_restaurant` .
- Preajustes habilitados en el PdV: *Ajustes* → *Punto de Venta* → *Take out / Delivery / Members* → **Presets** (`pos.config.use_presets = True`).
- El PdV debe tener el modo restaurante activo (`module_pos_restaurant = True`) y al menos un piso con mesas; sin mesas los escenarios 1 y 3 no existen.
- Contenedor de desarrollo: `lfernandez_v19` (definido en `docker-compose.yml` como `0D00_DEVELOPER_v19`), Postgres en el host `odoo-db` .

2.3 Decisiones que deben estar cerradas antes de codificar

Del documento de viabilidad, sección 9. Las dos que **cambian el código**:

- 1. ¿Se aplica también el sentido inverso (mesa → venta directa)? → determina si `syncOriginPreset` actúa en ambas direcciones o sólo hacia mesa (una línea, sección 5.5).
- 2. ¿El cajero puede seguir cambiando el preajuste a mano? → si la respuesta es "no", se quita el `presetManuallySet` y se oculta el botón de preajuste.

3. Arquitectura de la solución



Regla única de decisión, implementada en un solo lugar:

```
preset = order.table_id ? config.table_preset_id : config.direct_sale_preset_id
```

Todo lo demás son **puntos de invocación** de esa regla en los momentos en que el origen de la orden nace o cambia.

4. Estructura de archivos del módulo

```
pos_preset_by_order_origin/
├── __init__.py
├── __manifest__.py
├── README.rst
├── models/
│   ├── __init__.py
│   ├── pos_config.py          # 2 campos nuevos + integridad de datos de sesión
│   ├── pos_preset.py          # dominio de carga al frontend
│   └── res_config_settings.py  # espejo de los campos en Ajustes
├── views/
│   └── res_config_settings_views.xml
├── static/
│   ├── src/
│   │   └── app/
│   │       ├── models/
│   │       │   ├── pos_order.js    # uiState.presetManuallySet
│   │       │   └── services/
│   │       │       └── pos_store.js  # la lógica: 4 puntos de enganche
│   │   └── tests/
│   │       ├── tours/
│   │       │   └── preset_by_origin_tour.js
│   │       └── unit/
│   │           └── preset_by_origin.test.js
├── tests/
│   ├── __init__.py
│   └── test_preset_by_origin.py      # arranca el tour
└── i18n/
    └── es_D0.po                     # generado con tools/i18n-generator, nunca a mano
```

No hay modelos nuevos → **no hace falta** security/ir.model.access.csv .

5. Código

5.1 __manifest__.py

```
{
    "name": "POS Preset by Order Origin",
    "version": "19.0.1.0.0",
    "category": "Sales/Point of Sale",
    "summary": "Assign the POS order preset automatically from the order origin: "
    "table orders get the dine-in preset, direct sales get the takeaway preset, "
    "and a direct sale moved to a table is switched to the dine-in preset.",
    "author": "INDEXA SRL.",
    "website": "https://www.progressa.group/",
    "depends": ["pos_restaurant"],
    "data": [
        "views/res_config_settings_views.xml",
    ],
    "assets": {
        "point_of_sale._assets_pos": [
            "pos_preset_by_order_origin/static/src/**/*",
        ],
        "web.assets_tests": [
            "pos_preset_by_order_origin/static/tests/tours/**/*",
        ],
        "web.assets_unit_tests": [
            "pos_preset_by_order_origin/static/tests/unit/**/*",
        ],
    },
    "license": "Other proprietary",
    "installable": True,
}
```

`depends = ["pos_restaurant"]` no es opcional: garantiza que nuestros `patch()` se apliquen **después** de los de `pos_restaurant` (los assets de un módulo dependiente se cargan después), que es lo que permite envolver `setTable` y `prepareOrderTransfer` con `super`.

5.2 models/pos_config.py

```
from odoo import api, fields, models

class PosConfig(models.Model):
    _inherit = "pos.config"

    table_preset_id = fields.Many2one(
        "pos.preset",
        string="Table Preset",
        help="Preset applied automatically when the order is created on a table "
        "or moved to a table.",
    )
    direct_sale_preset_id = fields.Many2one(
        "pos.preset",
        string="Direct Sale Preset",
        help="Preset applied automatically when the order is created without a table.",
    )

    @api.depends("table_preset_id", "direct_sale_preset_id")
    def _compute_local_data_integrity(self):
        # El core marca `last_data_change` cuando cambia `default_preset_id`; los dos
        # campos nuevos deben provocar la misma invalidación para que las sesiones ya
        # abiertas vuelvan a leer la configuración.
        return super()._compute_local_data_integrity()

    def write(self, vals):
        res = super().write(vals)
        # Ambos preajustes deben poder seleccionarse en la interfaz del PdV.
        for config in self:
            presets = config.table_preset_id | config.direct_sale_preset_id
            missing = presets - config.available_preset_ids
            if config.use_presets and missing:
                config.available_preset_ids |= missing
        return res
```

Nota sobre `_compute_local_data_integrity` : en Odoo 19 el `@api.depends` de un `compute` sobrescrito **reemplaza** el del padre, no lo suma. Hay que repetir la lista completa del core (`point_of_sale/models/pos_config.py:626-628`) más los dos campos nuevos, o bien declarar el `depends` con `@api.depends(*PosConfig._compute_local_data_integrity._depends, ...)` . La forma segura y explícita es copiar la lista:

```
@api.depends('use_pricelist', 'pricelist_id', 'available_pricelist_ids', 'payment_method_ids',
              'limit_categories', 'iface_available_categ_ids', 'module_pos_hr',
              'module_pos_discount', 'iface_tipproduct', 'default_preset_id',
              'module_pos_appointment', 'cash_rounding', 'rounding_method',
              'only_round_cash_method',
              'table_preset_id', 'direct_sale_preset_id')
def _compute_local_data_integrity(self):
    return super()._compute_local_data_integrity()
```

5.3 models/pos_preset.py — paso crítico

```
from odoo import api, models

class PosPreset(models.Model):
    _inherit = "pos.preset"

    @api.model
    def _load_pos_data_domain(self, data, config):
        domain = super()._load_pos_data_domain(data, config)
        # El core carga sólo `available_preset_ids + default_preset_id`
        # (point_of_sale/models/pos_preset.py). Si los preajustes por origen no
        # están en esa lista, el many2one llega al frontend como un id colgante y
        # `config.table_preset_id` queda `undefined`.
        extra_ids = (config.table_preset_id | config.direct_sale_preset_id).ids
        if not extra_ids:
            return domain
        return ["|", ("id", "in", extra_ids)] + list(domain)
```

Sin esto el módulo "no hace nada" de forma silenciosa: el registro relacionado no existe en el cliente y `getOriginPreset` devuelve `undefined`. Es el error más probable en la primera prueba.

5.4 models/res_config_settings.py + vista

```
from odoo import fields, models

class ResConfigSettings(models.TransientModel):
    _inherit = "res.config.settings"

    pos_table_preset_id = fields.Many2one(
        related="pos_config_id.table_preset_id", readonly=False
    )
    pos_direct_sale_preset_id = fields.Many2one(
        related="pos_config_id.direct_sale_preset_id", readonly=False
    )

views/res_config_settings_views.xml :
```

```

<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <record id="res_config_settings_view_form" model="ir.ui.view">
    <field name="name">res.config.settings.view.form.inherit.pos_preset_by_order_origin</field>
    <field name="model">res.config.settings</field>
    <field name="inherit_id" ref="point_of_sale.res_config_settings_view_form"/>
    <field name="arch" type="xml">
      <!-- El <div class="row"> que contiene el preajuste por defecto -->
      <xpath expr="//field[@name='pos_default_preset_id']/.." position="after">
        <div class="row" invisible="not pos_module_pos_restaurant">
          <label for="pos_table_preset_id" class="col-lg-3" string="En mesa"/>
          <field name="pos_table_preset_id"
            options="{ 'no_create': True}"
            domain="[('id', 'in', pos_available_preset_ids)]"/>
        </div>
        <div class="row" invisible="not pos_module_pos_restaurant">
          <label for="pos_direct_sale_preset_id" class="col-lg-3" string="Venta directa"/>
          <field name="pos_direct_sale_preset_id"
            options="{ 'no_create': True}"
            domain="[('id', 'in', pos_available_preset_ids)]"/>
        </div>
      </xpath>
    </field>
  </record>
</odoo>

```

El <setting string="Take out / Delivery / Members"> del core **no tiene id** , por eso el xpath se ancla al campo pos_default_preset_id y no al bloque. Si Odoo le pone un id en una versión futura, conviene migrar el xpath a ese id .

5.5 static/src/app/services/pos_store.js — la lógica

```
import { patch } from "@web/core/utils/patch";
import { PosStore } from "@point_of_sale/app/services/pos_store";

patch(PosStore.prototype, {
  /**
   * Preajuste que corresponde al origen de la orden.
   * @param {Object|false} table registro restaurant.table (o falsy en venta directa)
   */
  getOriginPresetForTable(table) {
    if (!this.config.use_presets || !this.config.module_pos_restaurant) {
      return false;
    }
    const preset = table ? this.config.table_preset_id : this.config.direct_sale_preset_id;
    return preset || false;
  },

  /**
   * Alinea el preajuste de la orden con su origen actual.
   * No hace nada si el cajero ya eligió el preajuste a mano.
   */
  syncOriginPreset(order = this.getOrder()) {
    if (!order || order.finalized || order.uiState.presetManuallySet) {
      return;
    }
    const preset = this.getOriginPresetForTable(order.table_id);
    if (!preset || order.preset_id?.id === preset.id) {
      return;
    }
    order.setPreset(preset);
    if (typeof order.id === "number") {
      // Orden ya sincronizada: marcarla para que el cambio llegue a los demás equipos.
      this.addPendingOrder([order.id]);
    }
  },

  /**
   * @override
   * Escenarios 1 y 2: el preajuste se decide por la presencia de `table_id` en `data`.
   * Se pasa `preset_id` dentro de `data` para saltarse el `selectPreset(default_preset_id)`
   * del core (que abriría diálogos), y se aplica `setPreset` para que la tarifa y la
   * posición fiscal del preajuste queden aplicadas igual que en el flujo nativo.
   */
  createNewOrder(data = {}) {
    const preset = !data["preset_id"] && this.getOriginPresetForTable(data["table_id"]);
    const order = super.createNewOrder(preset ? { ...data, preset_id: preset } : data);
    if (preset) {
      order.setPreset(preset);
    }
    return order;
  },

  /**
   * @override
   * Escenario 3a: `setTable` puede "adoptar" una venta directa vacía y pegarle la mesa
   * (pos_restaurant/.../pos_store.js:665-692).
   */
  async setTable(table, orderUuid = null) {
    const res = await super.setTable(...arguments);
    this.syncOriginPreset(this.getOrder());
    return res;
  }
});
```

```

    },

    /**
     * @override
     * Escenario 3b: cuando la mesa destino está vacía, el core muda la orden y devuelve
     * `false`, cortando el flujo antes de llegar a `setTable`
     * (pos_restaurant/.../pos_store.js:905-921).
     */
    async prepareOrderTransfer(order, destinationTable) {
      const res = await super.prepareOrderTransfer(...arguments);
      if (!res) {
        this.syncOriginPreset(order);
      }
      return res;
    },

    /**
     * @override
     * Cualquier cambio de preajuste que pase por la interfaz se considera decisión del
     * cajero y desactiva el automatismo para esa orden.
     */
    async selectPreset(preset = false, order = this.getOrder()) {
      const previous = order?.preset_id;
      const res = await super.selectPreset(...arguments);
      const current = this.getOrder();
      if (current && current.preset_id && current.preset_id !== previous) {
        current.uiState.presetManuallySet = true;
      }
      return res;
    },
  });

```

Si el cliente NO quiere el sentido inverso (mesa → venta directa vuelve a *Para llevar*), cambiar `getOriginPresetForTable` por:

```

// Sólo se automatiza el lado "mesa"; la venta directa mantiene su preajuste.
return table ? this.config.table_preset_id || false : false;

```

...y en `createNewOrder` seguir pasando el preajuste de venta directa sólo cuando no hay mesa (es decir, mantener dos helpers: uno para creación, otro para transiciones).

5.6 static/src/app/models/pos_order.js

```

import { PosOrder } from "@point_of_sale/app/models/pos_order";
import { patch } from "@web/core/utils/patch";

patch(PosOrder.prototype, {
  initState() {
    super.initState();
    // El cajero cambió el preajuste a mano: no volver a corregirlo automáticamente.
    this.uiState.presetManuallySet = false;
  },
});

```

`uiState` se serializa con la orden (`related_models/index.js`), así que la bandera sobrevive a un refresco del navegador y a la sincronización entre equipos.

5.7 __init__.py

```
# pos_preset_by_order_origin/__init__.py
from . import models

# pos_preset_by_order_origin/models/__init__.py
from . import pos_config
from . import pos_preset
from . import res_config_settings
```

6. Puntos de integración con el core (mapa de referencia)

#	Qué se toca	Archivo del core	Motivo
1	createNewOrder	point_of_sale/static/src/app/services/pos_store.js:1398	el core asigna default_preset_id sin mirar data.table_id
2	setTable	pos_restaurant/static/src/app/services/pos_store.js:665	adopta ventas directas vacías y les pega la mesa
3	prepareOrderTransfer	pos_restaurant/static/src/app/services/pos_store.js:905	mueve la orden a una mesa vacía y corta el flujo
4	selectPreset	point_of_sale/static/src/app/services/pos_store.js:2471	detectar elección manual
5	PosOrder.initState	point_of_sale/static/src/app/models/pos_order.js:69	declarar la bandera en uiState
6	pos.preset._load_pos_data_domain	point_of_sale/models/pos_preset.py:39	cargar los presets nuevos al frontend
7	pos.config._compute_local_data_integrity	point_of_sale/models/pos_config.py:626	invalidar la configuración en sesiones abiertas

Lo que **no** hay que tocar:

- mergeOrders / transferOrder (pos_restaurant/.../pos_store.js:216–297, 923): cuando la mesa destino ya tiene orden, las líneas se funden en la orden destino, que **ya** tiene el preajuste de mesa. Sólo hay que verificarlo con una prueba.
- pos.order en Python: el preajuste llega desde el cliente en el preset_id del payload; no hace falta lógica servidor.
- Numeración fiscal (NCF/e-CF), l10n_do_accounting , l10n_do_pos : el preajuste no participa.

6.1 Cómo llegan los campos nuevos al frontend

pos.config **no** define _load_pos_data_fields , así que el mixin devuelve [] y _load_pos_data_read hace records.read([]) → **todos** los campos almacenados. Las relaciones se generan en pos.session._load_pos_data_relations (point_of_sale/models/pos_session.py:105), que con la lista de campos vacía omite únicamente los campos manual .

Consecuencias prácticas:

- Los dos `Many2one` nuevos viajan al PdV **sin registrar nada extra**.
- Un campo creado con **Studio** (`manual=True`) **no** llegaría al frontend: esta funcionalidad no se puede hacer con Studio, hace falta el módulo.
- El registro `pos.preset` apuntado tiene que estar cargado → punto 6 de la tabla anterior.

7. Configuración de datos (obligatoria, sin ella el módulo se comporta "raro")

Para cada Punto de Venta del cliente:

Ajuste	Valor
<i>Presets</i> (<code>use_presets</code>)	activado
<i>Available</i> (<code>available_preset_ids</code>)	Para llevar, Comer en el local
<i>Default</i> (<code>default_preset_id</code>)	Para llevar
<i>En mesa</i> (<code>table_preset_id</code>)	Comer en el local
<i>Venta directa</i> (<code>direct_sale_preset_id</code>)	Para llevar

`default_preset_id = Para llevar` no es cosmético: el filtro de `setTable` (`pos_restaurant/.../pos_store.js:682`) sólo adopta una venta directa vacía si su preajuste es el predeterminado. Con otro valor quedan órdenes vacías huérfanas en las pestañas.

En cada preajuste (*Punto de Venta* → *Configuración* → *Preajustes*):

Campo	Para llevar	Comer en el local	Por qué
<i>Identification</i>	Not required	Not required	<code>identification = "name"</code> abre un diálogo de nombre en cada orden (así viene el preset <i>Takeout</i> de fábrica, <code>pos_restaurant/data/scenarios/restaurant_preset.xml:24</code>)
<i>Manage orders by time</i> (<code>use_timing</code>)	desactivado	desactivado	activado pide franja horaria
<i>Guest</i> (<code>use_guest</code>)	desactivado	a decidir con el cliente	activado pide número de comensales al entrar a la mesa
<i>Pricelist</i>	vacío o la misma	la misma	evita re-precio al trasladar (sección 8.1)
<i>Fiscal Position</i>	vacío o la misma	la misma	idem
<i>Return mode</i> (<code>is_return</code>)	desactivado	desactivado	invertiría cantidades

Ajuste relacionado que conviene revisar: *Interfaz* → “*Select the table first or after registering the order*” (`pos.config.default_screen` , de `pos_restaurant`). Con “after registering the order” el PdV abre en pantalla de productos y **toda** orden nace como venta directa → el escenario 3 se vuelve el camino habitual, no la excepción.

8. Riesgos técnicos a controlar durante la implementación

8.1 setPreset re-aplica tarifa y posición fiscal

point_of_sale/static/src/app/models/pos_order.js:200 :

```
setPreset(preset) {  
    this.setPricelist(preset.pricelist_id || this.config.pricelist_id);  
    this.fiscal_position_id = preset.fiscal_position_id || this.config.default_fiscal_position_id;  
    this.preset_id = preset;  
    ...  
}
```

Si los dos preajustes tienen tarifas/posiciones fiscales distintas, trasladar una venta directa **ya capturada** a una mesa recalcula precios de las líneas existentes. Verificar con la prueba 4 de la matriz (totales antes/después). Si el cliente necesitara tarifas distintas y **no** quiere re-preciar, la alternativa es escribir `order.preset_id = preset` sin `setPreset` — pero entonces el preajuste queda desalineado de la tarifa, lo que debe quedar por escrito.

8.2 Se salta selectPreset a propósito

`createNewOrder` usa `setPreset`, no `selectPreset`, para no abrir diálogos. Si en el futuro alguien configura un preajuste con `identification` o `use_timing`, la orden se creará **sin** nombre ni franja: el core lo sigue bloqueando antes de cobrar mediante `presetRequirementsFilled` (`pos_order.js:174–195`), así que no se pierde información, pero el mensaje aparece más tarde en el flujo. Documentarlo en el README del módulo.

8.3 Pantalla de preparación (cocina)

`pos_order.js:272` guarda `last_order_preparation_change.sittingMode = preset_id`. Cambiar el preajuste **después** de enviar a cocina genera una diferencia que la pantalla puede mostrar como cambio de la orden. Si molesta, condicionar `syncOriginPreset` a que la orden no tenga impresiones previas:

```
if (order.uiState.lastPrints?.length) {  
    return;  
}
```

8.4 Multi-equipo

Sin `addPendingOrder`, un cambio de preajuste sobre una orden ya sincronizada se queda en el equipo local. Ya está en el código de la sección 5.5; probarlo con dos pestañas/sesiones (prueba 8).

8.5 Orden de los patch()

Los `patch()` de `pos_restaurant` sobre `PosStore.prototype` y los nuestros conviven por cadena de prototipos: `super` en nuestro `patch` llama al de `pos_restaurant`. Esto sólo funciona si nuestros `assets` se cargan después → `depends: ["pos_restaurant"]`. Si alguien cambia la dependencia a `point_of_sale`, `super.setTable` no existirá.

8.6 Mantenimiento entre versiones

Se parchean métodos internos del frontend. En cada migración mayor hay que revisar que `createNewOrder`, `setTable`, `prepareOrderTransfer` y `selectPreset` sigan existiendo con esa firma. Presupuestar 2-3 h por migración.

9. Pruebas

9.1 Test unitario (Hoot) — rápido, sin navegador

static/tests/unit/preset_by_origin.test.js :

```
import { test, expect } from "@odoo/hoot";
import { setupPosEnv } from "@point_of_sale/./tests/unit/utils";
import { definePosModels } from "@point_of_sale/./tests/unit/data/generate_model_definitions";

definePosModels();

test("una orden sin mesa toma el preajuste de venta directa", async () => {
  const store = await setupPosEnv();
  store.config.use_presets = true;
  store.config.direct_sale_preset_id = store.models["pos.preset"].get(1);
  store.config.table_preset_id = store.models["pos.preset"].get(2);

  const order = store.addNewOrder();
  expect(order.preset_id.id).toBe(1);
});

test("al pasar la orden a una mesa se aplica el preajuste de mesa", async () => {
  const store = await setupPosEnv();
  store.config.use_presets = true;
  store.config.direct_sale_preset_id = store.models["pos.preset"].get(1);
  store.config.table_preset_id = store.models["pos.preset"].get(2);

  const order = store.addNewOrder();
  order.update({ table_id: store.models["restaurant.table"].getFirst() });
  store.syncOriginPreset(order);
  expect(order.preset_id.id).toBe(2);
});
```

(El patrón de setupPosEnv / definePosModels está en
point_of_sale/static/tests/unit/models/pos_preset.test.js ; los datos de prueba de presets en
point_of_sale/static/tests/unit/data/pos_preset.data.js y
pos_restaurant/static/tests/unit/data/pos_preset.data.js ; las mesas de prueba en
pos_restaurant/static/tests/unit/data/restaurant_table.data.js .)

9.2 Tour JS + test Python — cubre los tres escenarios reales

static/tests/tours/preset_by_origin_tour.js (esqueleto; los helpers salen de
pos_restaurant/static/tests/tours/floor_screen_tour.js):

```

import { registry } from "@web/core/registry";
import * as Chrome from "@point_of_sale/./tests/pos/tours/utils/chrome_util";
import * as ChromeRestaurant from "@pos_restaurant/./tests/tours/utils/chrome";
import * as FloorScreen from "@pos_restaurant/./tests/tours/utils/floor_screen_util";
import * as ProductScreen from "@point_of_sale/./tests/pos/tours/utils/product_screen_util";

registry.category("web_tour.tours").add("PresetByOriginTour", {
  steps: () =>
    [
      Chrome.startPoS(),
      // Escenario 1: orden nueva desde una mesa
      FloorScreen.clickTable("2"),
      ProductScreen.presetIs("Comer en el local"), // helper propio del módulo
      Chrome.clickMenuOption("Orders"),           // volver y crear venta directa
      // Escenario 2: venta directa
      ChromeRestaurant.clickNewOrder(),
      ProductScreen.presetIs("Para llevar"),
      // Escenario 3: venta directa con líneas → mesa
      ProductScreen.addOrderline("Coca-Cola", "1"),
      ChromeRestaurant.clickTableButton(),
      FloorScreen.clickTable("4"),
      ProductScreen.presetIs("Comer en el local"),
    ].flat(),
});

```

tests/test_preset_by_origin.py :

```

import odoo.tests
from odoo.addons.pos_restaurant.tests.test_frontend import TestFrontendCommon

@odoo.tests.tagged("post_install", "-at_install")
class TestPresetByOrigin(TestFrontendCommon):
    def test_preset_by_origin_tour(self):
        takeaway = self.env.ref("pos_restaurant.pos_takeout_preset")
        dine_in = self.env.ref("pos_restaurant.pos_takein_preset")
        # Preajustes silenciosos: sin identificación ni franjas horarias.
        (takeaway | dine_in).write({"identification": "none", "use_timing": False})
        self.pos_config.write({
            "use_presets": True,
            "available_preset_ids": [(6, 0, (takeaway | dine_in).ids)],
            "default_preset_id": takeaway.id,
            "direct_sale_preset_id": takeaway.id,
            "table_preset_id": dine_in.id,
        })
        self.pos_config.with_user(self.pos_admin).open_ui()
        self.start_pos_tour("PresetByOriginTour", login="pos_admin")

```

Firma y clase base verificadas en el core: TestFrontendCommon (pos_restaurant/tests/test_frontend.py:13) ya crea cls.pos_config con modo restaurante, pisos y mesas, y start_pos_tour(tour_name, login="pos_user") está en point_of_sale/tests/test_frontend.py:51 ; los usuarios pos_user y pos_admin existen en el common (point_of_sale/tests/test_frontend.py:86,97).

9.3 Matriz de pruebas manuales

#	Caso	Resultado esperado
1	Venta directa nueva	Para llevar, sin diálogos

#	Caso	Resultado esperado
2	Mesa libre → orden nueva	<i>Comer en el local</i>
3	Venta directa vacía → seleccionar mesa	<i>Comer en el local</i> , sin orden huérfana
4	Venta directa con líneas → mesa libre	<i>Comer en el local</i> , mismos totales
5	Venta directa con líneas → mesa ocupada	líneas fundidas, orden resultante <i>Comer en el local</i>
6	Cambiar preajuste a mano en una mesa, salir al plano y volver	conserva la elección manual
7	Orden en mesa enviada a cocina, luego traslado	sin cambios inesperados en la pantalla de preparación
8	Dos sesiones/equipos	el cambio de preajuste se refleja tras sincronizar
9	Cerrar sesión y facturar	sin impacto fiscal (NCF/e-CF normales)
10	PdV sin use_presets o sin modo restaurante	módulo inerte, comportamiento nativo

9.4 Comandos

```
cd /Users/luisfernandez/repos/dev_env_odoo_pro-19

CONTAINER=lfernandez_v19
DB=v19_pos_preset_origin
DBF="--db_host=odoo-db --db_port=5432 --db_user=odoo --db_password=odoo_password"

# Instalar el módulo en una DB nueva (sin demo)
docker exec "$CONTAINER" bash -lc "createdb -h odoo-db -U odoo $DB" # PGPASSWORD según .env
docker exec "$CONTAINER" bash -lc "odoo -c /etc/odoo/odoo.conf -d $DB $DBF \
-i pos_preset_by_order_origin --stop-after-init --workers=0 --max-cron-threads=0"

# Actualizar tras cambios en Python/XML (el JS sólo necesita recargar el navegador)
docker exec "$CONTAINER" bash -lc "odoo -c /etc/odoo/odoo.conf -d $DB $DBF \
-u pos_preset_by_order_origin --stop-after-init --workers=0 --max-cron-threads=0"

# Tests (tour + unitarios del módulo)
docker exec "$CONTAINER" bash -lc "odoo -c /etc/odoo/odoo.conf -d $DB $DBF \
-u pos_preset_by_order_origin --test-enable \
--test-tags=/pos_preset_by_order_origin --stop-after-init --workers=0"
```

Los flags \$DBF son obligatorios: el odoo.conf del contenedor no trae db_host y sin ellos el comando falla con un error de socket que despista.

Para probar a mano conviene un script setup_v19_pos_preset_origin.sh siguiendo el patrón de setup_v19_l10n_do_hr_payroll_attendance.sh : crea la DB, instala pos_preset_by_order_origin , configura compañía/PdV con modo restaurante, un piso con 4 mesas, los dos preajustes silenciosos, los tres campos de configuración y un par de productos. Estimado: 1 h.

10. Convenciones del repositorio

- Ubicación: odoo-pro/pos_preset_by_order_origin/ (raíz del repo, no en store-addons).
- Manifest: version 19.0.1.0.0 , author INDEXA SRL. , license Other proprietary , website https://www.progressa.group/ .
- README.rst del módulo con propósito, configuración y limitaciones conocidas (secciones 7 y 8).
- Traducciones: **nunca** escribir el .po a mano; generarlo con tools/i18n-generator/generate-po.sh y sólo traducir los msgstr .
- pre-commit run --all-files antes de commitear (no usar --files , borra requirements.txt).

- CI: el workflow `tests.yaml` descubre los módulos de `odoo-pro` con `get_modules(depth=1)` ; no hay lista que editar. Si el módulo no fuese instalable, sí habría que regenerar la sección `NOT INSTALLABLE ADDONS` de `.pre-commit-config.yaml` .
- Commits en inglés, formato Conventional Commits.

11. Plan de trabajo y estimado

Paso	Entregable	Horas
1	Módulo base: manifest, <code>__init__</code> , README.rst, assets	1.0
2	<code>pos_config.py</code> (2 campos + integridad) y <code>pos_preset.py</code> (dominio de carga)	1.5
3	<code>res_config_settings.py</code> + vista de Ajustes	1.0
4	<code>pos_store.js</code> : <code>createNewOrder</code> (escenarios 1 y 2)	1.5
5	<code>pos_store.js</code> : <code>setTable</code> + <code>prepareOrderTransfer</code> (escenario 3) y verificación de <code>mergeOrders</code>	2.5
6	<code>presetManuallySet</code> (<code>pos_order.js</code> + <code>selectPreset</code>) y <code>addPendingOrder</code>	1.5
7	Tests unitarios Hoot + tour JS + test Python	2.5
8	Script de DB de prueba, matriz manual, README y traducción es_DO	2.0
Total		13.5 h (rango 12–16 h)

Fuera de alcance: kiosco/autopedido (`pos_self_order`), sentido inverso mesa → venta directa (+1.5 h si se aprueba), cambios de tarifas o posiciones fiscales.

12. Checklist de entrega

- ☐ Decisiones 1 y 2 de la sección 2.3 confirmadas por escrito.
- ☐ Módulo instalable en DB limpia y en copia de la DB del cliente.
- ☐ Los dos preajustes llegan al frontend (`config.table_preset_id` definido en la consola del PdV).
- ☐ Matriz de 10 pruebas manuales ejecutada y firmada.
- ☐ Tour y tests unitarios en verde (`--test-tags=/pos_preset_by_order_origin`).
- ☐ Totales de la prueba 4 idénticos antes y después del traslado.
- ☐ `pre-commit run --all-files` limpio y CI en verde.
- ☐ README.rst con limitaciones (8.2, 8.3) y `i18n/es_DO.po` generado.
- ☐ Configuración aplicada en el PdV del cliente y capacitación al cajero (el automatismo no impide el cambio manual).

Anexo — Rutas del core usadas en este manual

Qué	Ruta:línea
<code>createNewOrder</code> (asignación del preajuste)	<code>odoo/addons/point_of_sale/static/src/app/services/pos_store.js:1398–1409</code>
<code>selectPreset</code> (con diálogos)	<code>odoo/addons/point_of_sale/static/src/app/services/pos_store.js:2471</code>
<code>addPendingOrder</code>	<code>odoo/addons/point_of_sale/static/src/app/services/pos_store.js:1467</code>
<code>setPreset</code> (tarifa + posición fiscal)	<code>odoo/addons/point_of_sale/static/src/app/models/pos_order.js:200</code>

Qué	Ruta:línea
presetRequirementsFilled (validación antes de cobrar)	odoo/addons/point_of_sale/static/src/app/models/pos_order.js:174
initState / uiState de la orden	odoo/addons/point_of_sale/static/src/app/models/pos_order.js:69-86
sittingMode (pantalla de preparación)	odoo/addons/point_of_sale/static/src/app/models/pos_order.js:272
Campos de preajuste del PdV	odoo/addons/point_of_sale/models/pos_config.py:154-156
_compute_local_data_integrity	odoo/addons/point_of_sale/models/pos_config.py:626-629
pos.preset._load_pos_data_domain	odoo/addons/point_of_sale/models/pos_preset.py:39
Carga de datos y relaciones al frontend	odoo/addons/point_of_sale/models/pos_session.py:105-186
setTable (adopción de venta directa)	odoo/addons/pos_restaurant/static/src/app/services/pos_store.js:665-692
setTableFromUi	odoo/addons/pos_restaurant/static/src/app/services/pos_store.js:743
prepareOrderTransfer	odoo/addons/pos_restaurant/static/src/app/services/pos_store.js:905-921
transferOrder / mergeOrders	odoo/addons/pos_restaurant/static/src/app/services/pos_store.js:923-941 , 216-297
isDirectSale / isFilledDirectSale	odoo/addons/pos_restaurant/static/src/app/models/pos_order.js:65-78
use_guest y presets maestros	odoo/addons/pos_restaurant/models/pos_preset.py
Preajustes de fábrica (Dine In / Takeout)	odoo/addons/pos_restaurant/data/scenarios/restaurant_preset.xml:20-30
Vista de Ajustes (sección de preajustes)	odoo/addons/point_of_sale/views/res_config_settings_views.xml:55-73
Ejemplo de tour restaurante	odoo/addons/pos_restaurant/static/tests/tours/floor_screen_tour.js
Ejemplo de test unitario Hoot	odoo/addons/point_of_sale/static/tests/unit/models/pos_preset.test.js